04/07/26 lesson 01 Augmented with AI

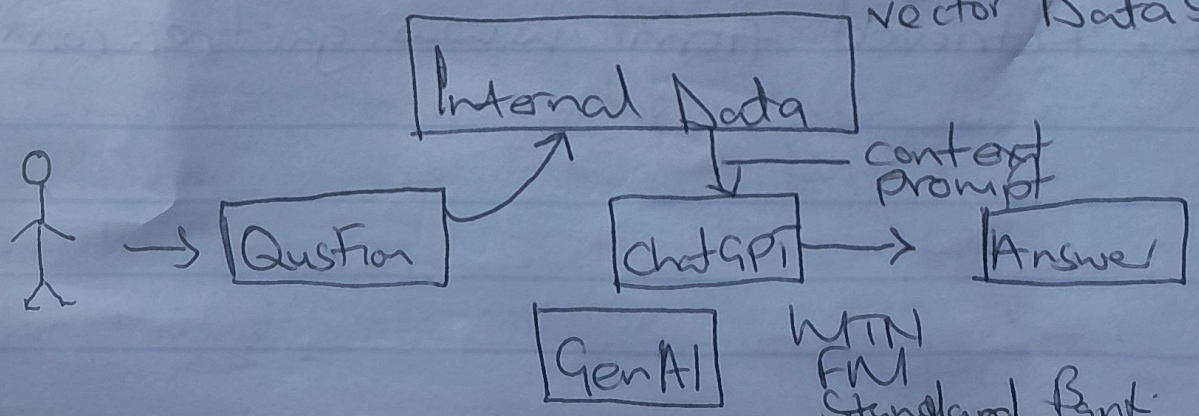
AI tools

- Gemini
- ChatGPT
- Claude
- Meta AI
- Grok
- deepseek
- Copilot
- NotebookLM
- Gemma
- Canva
- Perplexity
- Kimi
- base44
- replit
- names
- Canva
- lovable
- Vercel
- Cursor
- Lelapa AI
- kiro
- DELL-A
- Zapier
- Whisperflow
- SmartCAT
- Julius ai
- Quillbot
- Genie AI

GPT - Generative Pre-Trained Transformer
LLM - Large Language Model
Machine learning -> Enables a machine to learn patterns & learns patterns from data
Branch of Artificial intelligence (AI) where computers / systems learn patterns from data to make predictions

AI - Artificial intelligence;
for an AI system to exist it needs to learn patterns from data.

Hallucinating - make things up that are incorrect
RAG - Retrieval Augmented Generation



RAG It is a way to minimize hallucination.
we don't want chat GPT to lie

Prompt — a question you give to an AI
- instruction
- a command
- a guardrail
- a request

What is prompt Engineering
A way to construct an effective prompt to ensure clarity to the AI so that it does not give you the wrong answer
- It is a clear instruction, and detailed instruction, and specific instruction
- A GenAI tool requires a prompt in order to generate what you need.

Different ways of prompting
1. Few-shot prompting
You give the LLM few examples so that it understands your question
Is a way of interacting with a LLM (GenAI) by asking it question and give it examples as part of the context for it to be able to understand what want

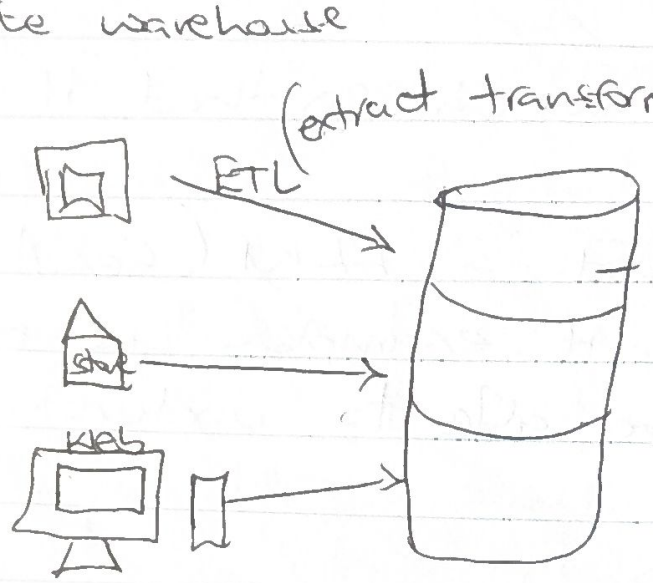
2. Zero shot prompting
This is when you ask a question without giving any example.
- Direct but no clarity

③ Contextual prompting
You give background information

④ Constraint - based prompting
to limit the response
- Restriction based
- Controlling the kind of output you want from the
- Limiting the scope of the output

Blog 86 How do we build AI Lesson 3 Augmented with AI

Machine Learning Process
- The outcome of a machine learning process in an AI to system
- In order to build a machine learning model - we need d
The role of the Data Engineer is to make the data available in the data warehouse.
Data Engineer Build data pipelines to store the data in the warehouse



Step ① Data Collection

Step ② Stakeholder : Define the problem
Before building AI/ML solution we need to define the prob
we are trying to solve

Bank - Predict the likelihood of a customer leaving my bank and going to another

Goal - we want to retain our customers

- Mark a machine learning model to predict the customers likely to churn

③ Explore and understand the data

- Exploratory Data Analysis (EDA)

Ask data driven Questions

↳ to analyze the data to understand patterns trends and relationships

↳ Which customers are leaving?

↳ Does age affect Churn?

↳ Do customers with many complaints leave more

④ Prepare the Data for machine learning

Feature Building

- Converting text into numbers (encoding)
- Scaling numeric values
- Creating new useful columns
- Splitting data into feature & forget

Feature Engineering

Feature Selection

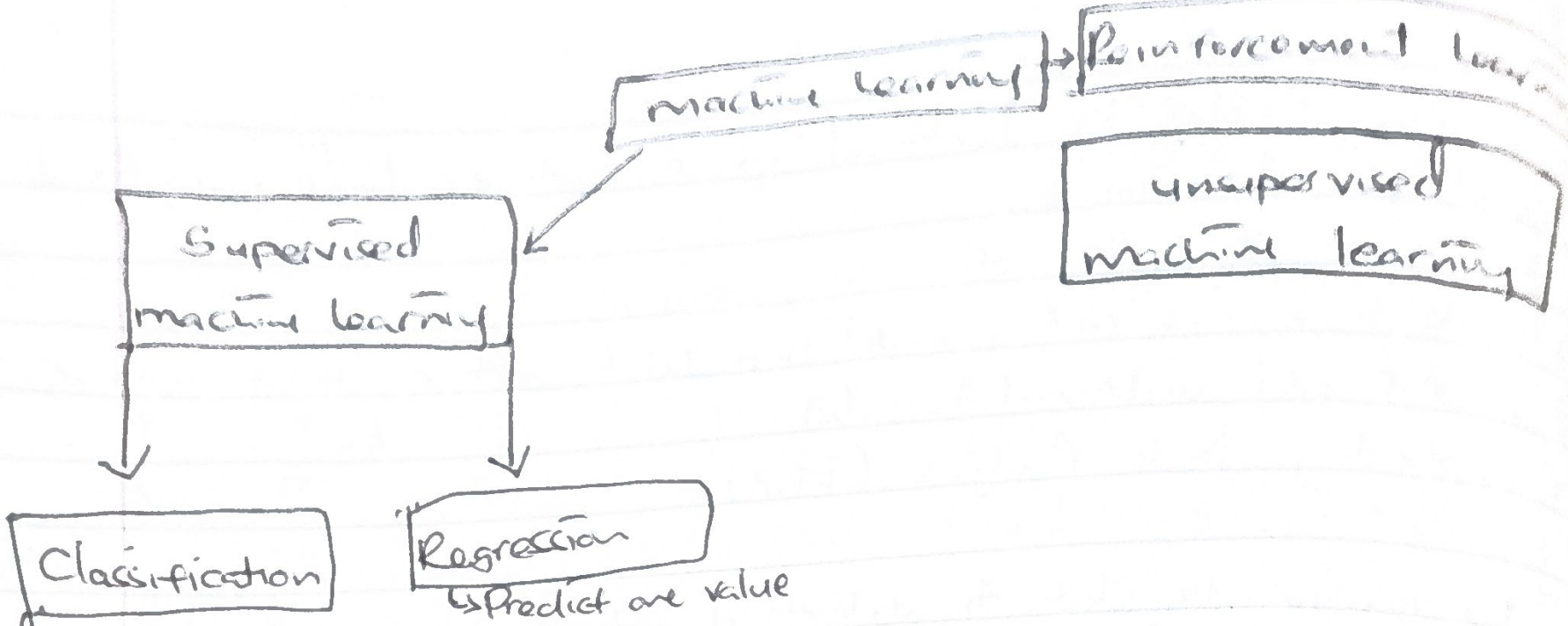
Training Data — used to teach the model 80%.

Testing Data — used to check if the model works (20%

1	5
2	6
3	7
4	8

9
10

⑥ Choose machine learning Algorithm



Lethabo	No
Mbali	Yes
Tumi	Yes
Thuto	Yes

Lesson 4:
Machine learning: Data Collection & Preparation
The Concept of Machine learning
AI Its an intelligence that is artificially generated
meaning that intelligent has to be trained from somewhere
- Somewhere is Data

How do we build our ways machine learning?
We build the machine learning
We build our way to machine learning by having data

Data Analytics product Dashboard, Tablue, Reports

* Whenever you purchase something you are generating data

The more you buy the more you are generating data.
Data is information that is structured, it has some

The hierarchy of Data
- Information -> lot of Data that does not make sense
- Data -> Information cleaned and becomes data
- knowledge -> Now you have data that is useful to make stuff
- Wisdom -> Ability to apply knowledge

- Database is the place we get transactional data.
- Database is where we place all transactional data as is
- (It is not cleaned or processed)
- Once it's cleaned or processed it goes to datawarehouse
- A Datawarehouse is where clean data is stored. (this is where the data is cleaned and formatted)
- Datawarehouse is used for analytical purpose and training machinery models

Software engineers designs, build, tests and maintains computer software eg website They use programming languages to create software

Software engineers are grouped into two:
Front-end and Backend
Frontend engineers build the parts of software that users see and interact with eg screens HTML, CSS, JavaScript
Backend engineers build the behind scenes systems. They work on servers, databases, security login systems & data processing Python, Java, SQL databases

ADVANCED SQL
Wildcard and Patterns

Select → output
From → Tables name
Where → Filter

Wildcards allows you to filter by matching patterns

Select name
from Student
Where CourseName = 'Analytics"

eg

Course
Data Analytics
Business Analytics
Advanced Analytics

Mathematics
Engineering

like matches you patterns to what ever you Specify
like → finds values that match pattern
% - Match any number of characters
_ underscore
- Where CourseName like "% Analytics %"
- Where CourseName like "Analyti_s"

When ~~filtering~~ ~~after~~
Coursename besides Analytics
Where CourseName Not like "% Analytics %"

Name	Emails	Phone
John	Jman@gmail	27 723 4567
Jane	... @ Yahoo	37123 4567
Thabo	@ outlook.com	

Kllere emails like '%@gmail.com'

eg South African phone number
Kllere phone like '27 76 %'

eg Name starts with J
Kllere name like 'J%'

eg Name with 3 letters
Kllere name like '___'

Advanced Pattern matching

Regular Expressions : [ ] range of characters
Regex

eg Name starts with A, B, C
Kllere name like '[A-C]%'
→ 'A%', 'B%', 'C%'

eg Excluding name starts with X, Y, Z ('%[X-Z]%')
→ '%Y%', '%X%', '%Z%'

Kllere name Not like '%[X-Z]%'

Kllere name in ('Thabo', 'John')

eg Multiple conditions
Kllere name like Any ('A%', 'B%', 'C%'

* Kllidcards are the specific symbols (like % or _) that represent unknown or variable characters in a search string
* Patterns is the entire string passed to the like operator
- It includes both regular characters and wildcard characters

Window Function

- Performs a calculation across related rows without collapsing them or losing details

Function() over (partition by column order by column) As calculation

Sum(sales) over (partition by transaction_date order by date) As running_total

(1) How is each store performing over time?

- Partition to split data into min datasets

~~How is each store~~

- Partition groups rows in a dataset based on certain column

store_location, date, sales

mini datasets per store location

(2) What are the best performing products

Select

product_type,

Sum(transaction_qty*unit_price) As sales,

Rank() over (order by sum(transaction_qty*unit_price) desc) As product_rank

From brightlearn.bright coffee.shop_sales

WHERE

Group By product_type

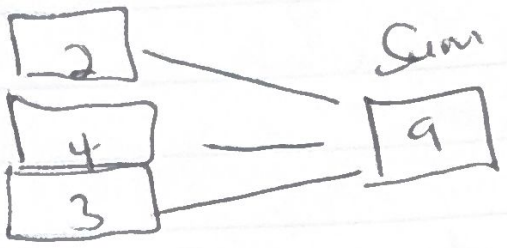
Window Functions

Aggregate
AVG(), SUM()
MIN(), MAX()
COUNT()

Rank
Rank ()
dense rank ()
NTile ()

Value
lag ()
Lead ()

① Aggregation



Running total using window fun



Number	tradition
2	
4	
3	
	9

Window Function Sum
2 — 9
4 — 9
3 — 9

~~Characteristics of aggregate window function~~

```
SELECT
  column,
  Fuction (column) over (partition by grouping column
                         order by sort_column)
FROM table
GROUP BY column ;
```

sum()
AVG()
....

Characteristics of aggregate window functions
① Operates within a window
② Maintain row level details

Date	Seller	Amount	Running_Total (order by)	Seller_Total
Jan 1	John	500	500	1400
Jan 5	John	600	1100	1400
Jan 9	John	300	1400	1400
Jan 2	Ben	700	700	1400
Jan 6	Ben	800	1500	1900
Jan 8	Ben	400	1900	1900

```
SELECT  Date,
        Seller,
        Amount,
        SUM(Amount) over (partition by Seller
                          Order by Date) As running_T
FROM  Sales
GROUP BY  Date,
          Seller,
          Amount;

SUM (AMOUNT) over ( partition by SELLER) As Seller_Tota
FROM  Sales
GROUP BY  Date,
          Seller
          Amount;
          )
```

2 Ranking Window function
- Assign ranking or row_number to specific rows
- Rank ( ), Dense_rank ( ), row_number ( )
- Assigns a unique rank to each distinct row within the sam partition

E.

Name	Dept	Salary	Rank	Dense_rank	row_number
Alice	Eng	9000	1	1	1
Bob	Eng	8000	2	2	2
Carol	Eng	~~9000~~ 7000	~~4~~ 3	2	3
David	Eng	7000	3	3	4
Thabo	HR	6000	1	1	1
Frank	HR	5500	2	2	2
Frank	HR	5500	2	2	3

```
SELECT Name,
       Dept,
       Salary,
       Rank() over (partition by dept
                    order by salary desc) As Rank
```

Dense_rank
- Assign distinct rank
- Does

```
Dense_rank() over (partition by dept
                   Order by Salary desc)
```

Raw number

```
Select name, dept, Salary,
rank()
dense_rank()
raw_number() over. (partition by dept
                    order by salary) As raw_numb
From Employees
Group by name, dept, Salary
```

Shortcut

```
Select name, dept, salary,
rank ( ) over x As rank
dense_rank ( ) over x As dense_rank
row_number ( ) over x As row_number
Sum ( Salary) over x As
From Employee
Window x As (partition by dept order by salary desc)
```

key word ↑

Example 5

Customers

Customer_ID	Name	email	Row_number
C1	Alice	alic@gmail.c	1
C2	Bob	Bob@gmail.com	1
C3	Alic	alice@gmail.com	2
C2	Frank	Frank@gmail.com	1

```
Select customer_ID,
       row_number( ) over (partition by nam, email
                           order by customer_ID ASC
                           ) As row_num
From Coactomers ;
```

Value Window function

Lag ( ) - allows access to previous

lead ( ) - retrieves the value from the next row

Month	Revenue	Lag / Prev_month	Month on Month / MoM_change	Lead / Next...
Jan	10 000	0	+ 10 000	12 ...
Feb	12 000	10 000	+ 2000	9 5...
Mar	9500	12 000	- 2500	14 ...
April	14 000	9500	+ 4500	null
		~~14000~~		

```
Select month, revenue
Lag (revenue) over (order by month) as Prev_month
Lead (revenue) over (order by month) as next_month
revenue_lag(revenue) over (order by month) MOM_cha
From monthly_revenue;
```

First_value ( ) : retrieve the value of a specified column from the first row within the window

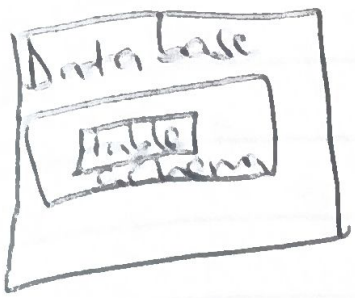
Example

Employee	Month	Revenue	Starting Rev	Last Rev
Alice	Jan	1000	1000	1200
Alice	Feb	1500	1000	1200
Alice	March	1200	1000	1200
Bob	Jan	800	800	2000
Bob	Feb	2000	800	2000

Databases and tables (DDL fundamentals)

SQL Commands

SQL, commands



DML -
Data manipulation La
- used to manipulate data
* Insert
Update
Delete

DDL -> Data Definition Language

- Define
Alter
Delete
In DDL you will find commands like:
* Create
Drop
Alter

Database - collection of organised data
Table - store data in rows and columns
Row - Record (A single entry of data in a table. Column (field)
Attribute of the data.
Primary Key - uniquely identifies each record

fact table
Data always changing

Sales
transaction_ID (Primary key)
Date
Product_ID (foreign key)
Customer_ID
QTY
transaction ID & table: uniquely identifies the row.

Dimension table
Data does not change (constant)

Product
Product_ID (Primary Key)
Name
Price
Description

Select *
From sales A
Join product B on
ON A.productID = B.productID

Q4 The difference between bigint (BIGINT) and int (I
is that
used for large scale numbers
- BIGINT can carry large values while INT carries sm
amount of values.
- Varchar specify the number of characters.
- DELTA is used to change the structure of an exist
table. Modify an existing table
Add a new column
Remove a column
Change a column's data type
Rename a column

: ALTER TABLE my. coffee. table. dim_customers RENAME
loyalty_points to points

Before renam
NB/// ALTER TABLE my. coffee table. dim_customers SET TBLPROPERTIES
('delta. columnMapping. mode' = 'name')

Data Definition Language

Commands that define the structure of your database

- Database is a collection of related tables, views, queries

create database name_of_database;

use name_of_database;

Drop database if exists

- Tables stores data in raw and colums
- Row is an entry in a table
- Column stores one type of information

Example: Employee

columns ↓

ID	Name	Gender	Age
1	John	Male	28
2	Mary	female	24
3	Thabo	Male	43

Create table table_nam(
Column_name1 datatype [constraints]
Column_name2 datatype [constraints] [default]

Data Types

Textual data types

Char (size) - Fixed length (gender - [M, F])

Varchar (size) - variable length string (name description)

Numeric Data types

INT → stores whole number (positive, Negative; 0)

BIGINT → stores large whole numbers (ID, phone number)

FLOAT → Stores approximate decimal numbers (single precision)

DOUBLE → stores approximate decimal numbers (double precision)

- Decimal → stores exact decimal number with fixed precision
  Decimal (M, d) → Decimal (8,2)
  M = total Digits
  D = Decimal places

Data and Time data types
Year → store a year value
Date → store date in format (YYYY-MM-DD)
Time → store time in format (HH:MM:SS)
Datetime → store both the date and time (DATETIME)
Timestamp → Often used to store created time
Useful for tracking insert time

Column Constraints
- Not null → value required
Unique → all values different
Default → use present values if no value is given
primary key → uniquely identifies the each row
Auto-Increment → value increases automatically for each row

Example: create table sales
Event_Date Timestamp (not null) ~~Default Now()~~
Default Now()

Key and relationships
- Primary key uniquely identifies each row
- Foreign key → links one table to the primary key of another table

Employees
ID (PK)
Name
Gender
Dept_ID (FK)

Department
Dept ID (PK)
Dept_Name
Description

Changing tables with alter table

Rename table : Rename table old_name To new_name

Alter table : Add column
Drop Column
Modify Column
Add / Drop constraints

Example Alter : Alter table employees add column address;
multiple changes - Alter table employees
Drop column age
Add column salary float not null
Add column years_in_company INT not nul

Describe employees : used to check the structure of the table.

Data Manipulation Language
① Insert into : used to insert new records into a table
Insert into employees (name, gender, Salary)
Value (Prim, Female, 50 000

② Update ; used to modify existing records
- Where is very import
Update employees

Employees
ID (PK)
Name
Gender
Dept_ID (FK)

Department
Dept ID (PK)
Dept_Name
Description

Changing tables with alter table

Rename table : Rename table old_name To new_name.

Alter table : Add column
Drop Column
Modify Column
Add / Drop constraints

Example Alter : Alter table employees add column address;
multiple changes - Alter table employees
Drop column age
Add column salary float not null
Add column years_in_company INT not null

Describe employees; used to check the structure of the table.

Data Manipulation Language

① Insert into : used to insert new records into a table
Insert into employees (name, gender, Salary)
Value (Ram, Female, 50 000

② Update ; used to modify existing records
- Where is very import
Update employees

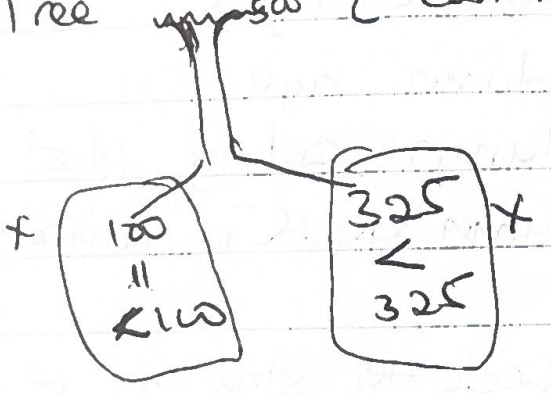
Set address = "New road, Gauteng"
Where id = 1;

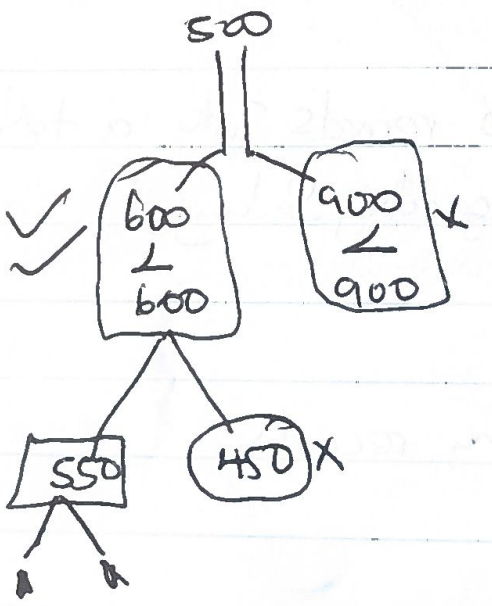
⑧ Delete : used to remove records from tables
Where is very important
Delete from employees
Where id = 3;

Indexes →
- Separate, sorted structure that lets the engine jump to row without scanning everything

No index (full scan)	with index (jump to row)

In Database we have (Indexes)
B-Tree ~~yyy500~~ { customer_ID < 500, from 10 million





we continue breaking it do
until we get the actual valu

create index idx_customer_id ON dim_customers(customer ID).

clustered vs Non-clustered

- points back to the original rows.
- like the back of the book index

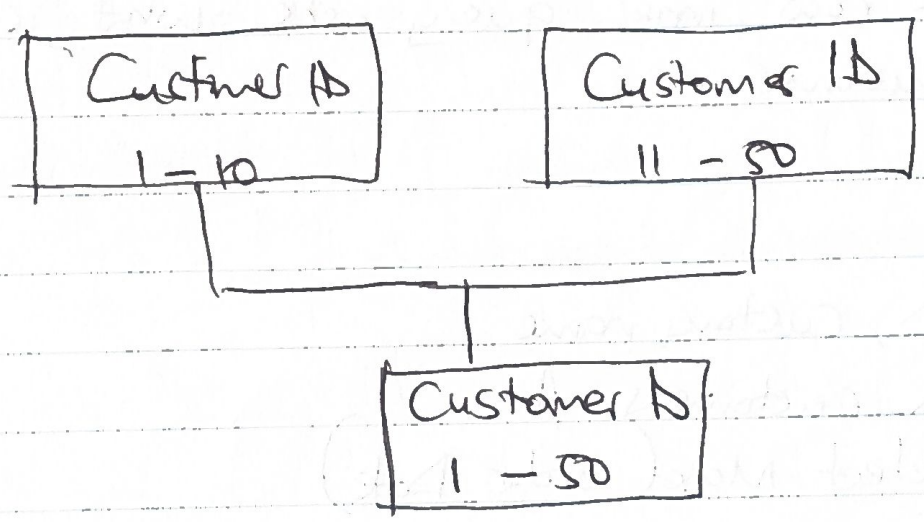
Physical reorders the rows of the table itself to match the index order

Databricks Toolkit

- Instead of index point to individual rows databricks works at a file level.
- Skips files that dont contain relevant data.

Partition by → Splits data into folders by column value

Optimize → Compacts small files



Z order → group related rows into the same files

So

Subqueries

Select statement nested inside another select

A

Select

Customer ID, Customer name, Customer age

from (select ~~customer ID~~ *

From dim_customer

Where Age > 25)

} Inner query runs first

As A
Where address = "New road"
~~Example 2~~

Example 2:
Select *
From (select distinct customer ID, name, address
From dim_customers) as A;

Correlated subquery
- Inner query references a column from the outer query
- runs once per row
-> Find customer whose most recent order was in 2025
For each row, inner query asks what is the max order for this customer

Select
Customer_ID, Customer_name
From dim_customers A
Where (Select Max(order_Date)
From fact_orders B
Where A.customers_ID = B.Customer_ID)
>= '2025-01-01';

Non-correlated subquery
- The inner query is independant
Runs once, returns values, list

Example: Find customers who spend above average
Step 1: what is the average
Step 2: filter by the average

Table Customers

Customer ID	Name	Total_spend
1	John	200
2	Mark	150
3	Thabo	100
4	Lerato	200

① Average spend ⇒ Avg = $\frac{\text{Total_Spend}}{\text{\# Customers}}$

= 650/4 → 162.5

② Filter by the average (John, Lerato)

```
Select customer ID, Name, Total_Spend
From Customers
Where Total_Spend > (select AVG (Total_spend)
                     from customers);
```

Correlated Subquery

Customer 1 → inner runs
Customer 2 → inner runs
Customer 3 → Inner runs
Slow on big tables

Non correlated
Inner query
runs once
↓
Outer filters all rows

CTEs
Common Table Expressions
- Gives you a way to name a temporary results set
- Define CTE using with

Define once and use it like a normal table

Basic Syntax

```
With cte_name As (
Select columns
From .....
Where .....
)
Select *
From CTE_Name
Where ..... ;
```

Single CTE

Example: Avg speed per product only where Avg above 300.

```
With product_Averages as (
 Select product,
Avg (total_spend) as avg_Spend
from customer
Group products
)
Select product, avg_spend
from product_averages
where Avg_spend > 300;
From product_averages
Where Avg_spend > 300;
```

Chaining 2 CTEs

- You define multiple CTEs
- Separate them with a comma
- Can reference the CTEs defined before it.
- Only on with at the beginning